

KubeBalancer

A Cloud-Native Image Classification
Pipeline on Kubernetes

PREPARED BY

Angeudy Brisita Perez

CHAPTER 1

Vision

1.1 Why KubeBalancer?

One of the many issues with image compilation nowadays is being able to optimize and resize images for both cost savings and performance improvements. The main idea behind KubeBalancer is to address said issue by providing a simple image classification pipeline built into Kubernetes. We can ensure that images go through some sort of linting process that will optimize image size and better allocate resources in the container. Images without any sort of pre-processing and optimization can easily range from 1 to 1.6 gigabytes. By opting to use KubeBalancer to handle the preprocessing and image optimization, we can downsize this image to roughly 5-150 megabytes depending on the image build we're packaging. This typically involves swapping to minimal base images, optimizing what files to keep (this usually involves building the product at some point in the pipeline and only keeping the built artifacts), and excluding unnecessary files by adding a `.dockerignore` and excluding heavier folders such as `node_modules`.

1.2 Goals

KubeBalancer's goal is to serve as an image optimization service that acts as a "balancer" of some sorts by rearranging images into their optimal sizes through its preprocessing service, which resizes and normalizes images. This then hands it over to the inference service, which, through a pretrained model, hands over the preprocessed image and results alongside the original image to a storage service that later persists it. The final architecture should aim to achieve the following 4 goals:

- 1) **Decomposition:** Implements the preprocessing capabilities, as well as inference and storage as independent services, which ideally should live in their own separate container images separated by a thin API gateway containing an interface and release cycle.
- 2) **Orchestration:** Encompasses the process of running said services on a multi-node Kubernetes cluster and relying on the platform's built-in capabilities to maintain replicas, self-heal failing services, ease to roll updates, and having horizontal autoscaling instead of handwritten scripts.
- 3) **Multi-container pod design:** Using containers and sidecars to solve the issue at hand rather than just being there.
- 4) **Evidence collection:** Measuring performance metrics, demonstrating recovery behavior under load and deliberately injected failures, basically backing every claim made about the architecture with data.

1.3 Architecture at 10,000 Feet

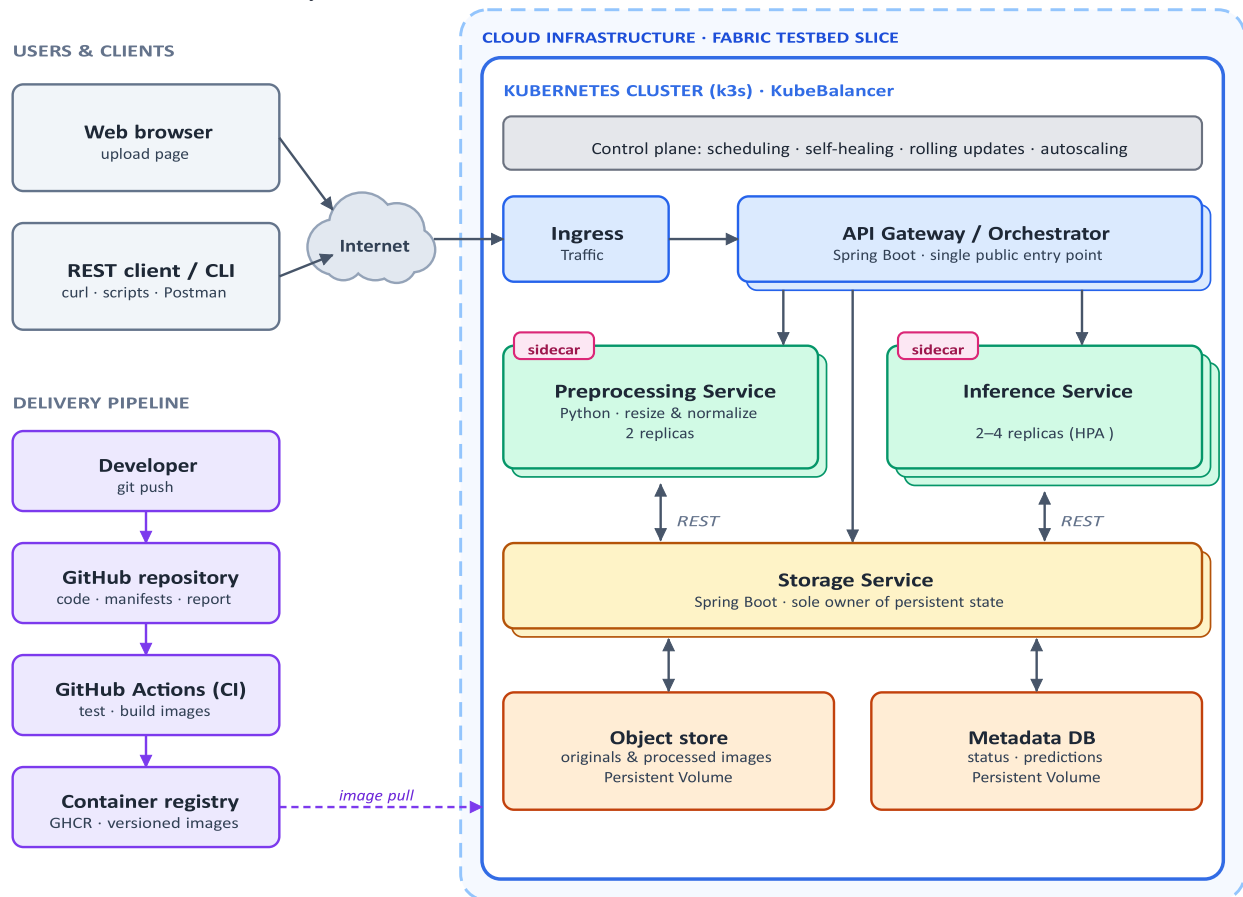


Figure 1. KubeBalancer at 10,000 feet. Stacked boxes denote replicated services; pink tabs denote sidecar containers that share a pod with the service they support.

Figure 1 shows the architecture from the outside in. On the left, users connect to KubeBalancer via a browser-based upload interface or REST client, and each connection goes over the network to one entry point, an Ingress controller sitting at the perimeter of the Kubernetes cluster, which routes requests to the API Gateway. The gateway is the only component that the clients see, and it directs each request through the process of having the Storage Service persist the image, the Preprocessing Service rescale and standardize it, and the Inference Service classify it using a pre-trained neural network and report the results. There is no inter-component communication through shared memory or files; the components communicate with one another via internal REST calls directed to fixed Kubernetes service names, and the durable storage is behind the Storage Service in two stateful backends: an object storage for image files and a document database for metadata and predictions, both residing in persistent volumes that survive pod crashes. The separation of state from the other parts of the system makes the latter fully disposable: the Kubernetes control plane maintains the desired number of replicas for each stateless service, restarting failed pods, scaling up inference pods as needed. The entire cluster runs on three virtual machines hosted in a FABRIC testbed slice, while the delivery pipeline on the bottom-left corner completes the loop from code to cluster: the cluster pulls versioned container images built and uploaded to Docker Hub by GitHub Actions based on commits to GitHub repository.

CHAPTER 2

Proposed Implementation

The implementation process will follow a bottom-up approach, whereby every subsequent part relies on a part that already works. Each component is implemented and unit-tested against a predefined RESTful interface contract, packaged as a container image, and deployed together with other components using Docker Compose to validate the complete workflow. The whole application is then ported to Kubernetes, first locally on a k3d [2] cluster and later a three-node k3s cluster [2] in a FABRIC slice [3]. Additional features such as sidecars, scaling, and automatic build are applied after establishing the basic functioning. Evaluation follows afterwards. The gateway and storage services are written in Java and utilize the Spring Boot framework, while preprocessing and inference use Python and FastAPI respectively. As for the classifier, it is a pre-trained MobileNetV2 model [4] serving in ONNX Runtime [5].

The project is divided into ten packages in three stages, which are shown in Table 1 below, with all the packages producing a deployable and demonstrable artifact in the end. Packages are not listed according to the time when they are worked on but rather following the dependency order. After defining the Storage Service API, there is no more connection between the preprocessing and inference services, thus they can be developed in parallel with CI pipeline at the same time.

Table 1. Work packages, dependencies and estimated effort.

#	Work package	Scope	Depends on	Estimate
Stage A: Foundation				
1	Cluster environment	FABRIC slice (3 VMs), scripted k3s install, local k3d	—	8–12 h
2	Storage Service	REST API for images and records;	—	12–16 h
Stage B: Pipeline services				
3	Preprocessing Service	Resize, center-crop, normalize; store image + tensor	2	6–8 h
4	Inference Service	top 5 prediction endpoint	2	8–10 h
5	API Gateway	Orchestration, input validation, retries, upload page	2–4	10–14 h
6	Containerization	Dockerfiles; full pipeline under Docker Compose	3–5	6–8 h
Stage C: Kubernetes, automation and validation				
7	Kubernetes deployment	Deployments, StatefulSets, Services, Ingress, probes	1, 6	12–16 h
8	Sidecars & autoscaling	Model-fetcher init container, logging sidecar, HPA	7	8–12 h
9	CI pipeline	GitHub Actions: test, build, publish images to GHCR	6	4–6 h

10	Testing & evaluation	Functional, load (k6) and failure-injection tests	7, 8	10–14 h
Total, core system				84–116 h

Time estimates are number of hours spent by one person and include testing and documentation of each module/package. Core system is estimated to be 84-116 hours, while 15% buffer time for testbed issues like slice availability has been reserved for it. Any add-ons like a messaging queue, monitoring, or service mesh are estimated to be 6-12 hours each and will be done only after the completion of core.

References

- [1] B. Burns and D. Oppenheimer, “Design Patterns for Container-based Distributed Systems,” in *Proc. 8th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud ’16)*, 2016.
- [2] K3s Documentation, “Lightweight Kubernetes.” [Online]. Available: <https://docs.k3s.io/>
- [3] I. Baldin et al., “FABRIC: A National-Scale Programmable Experimental Network Infrastructure,” *IEEE Internet Computing*, vol. 23, no. 6, 2019.
- [4] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [5] ONNX Runtime Documentation, Microsoft. [Online]. Available: <https://onnxruntime.ai/docs/>